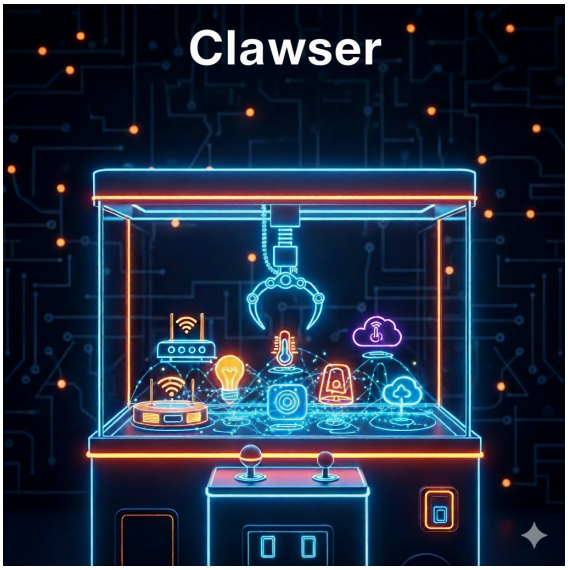
Department of Computer Science & Engineering
The University of Texas at Arlington

# Detailed Design Specification
# CSE 4317: Senior Design II
# Spring 2026




# Clawser
# IOT Claw Game

Josalyn Abbott, David Aviles
Carson Brown, Phillip Cook
Justine Talia Gacho, Jeff Hernandez
Dang Khang Lam, Jesus Martinez
Aryan Prasain, Jordan Taylor
Jeff Wright, Alexander Zacarias

## Revision History

| Revision | Date | Author(s) | Description |
|---|---|---|---|
| 0.1 | 1.20.2026 | DA | document creation |
| 0.2 | 2.2.2026 | DA, JT, JA, AZ, PC, JTG, JH, DL, JM, CB, JW, AP | complete draft |
| 1.0 | 05.01.2025 | AZ, DA, PC, JA, JT, JTG, JH, DL, JM, CB, JW, AP | Final Submission |

# Contents


**1 Introduction** **5**

**2 System Overview** **5**

**3 Cabinet Layer Subsystems** **6**
3.1 Layer Hardware . . . . . 6
3.2 Layer Operating System . . . . . 6
3.3 Layer Software Dependencies . . . . . 6
3.4 Power Supply Subsystem . . . . . 6
3.5 E-Stop Subsystem . . . . . 7
3.6 Camera Subsystem . . . . . 8
3.7 Computer Subsystem . . . . . 9
3.8 Light Controller Subsystem . . . . . 10
3.9 Microcontroller Unit Subsystem . . . . . 11
3.10 Prize Sensor Subsystem . . . . . 12
3.11 Gantry Motherboard Subsystem . . . . . 13
3.12 Claw Subsystem . . . . . 14

**4 Cloud Layer Subsystems** **16**
4.1 Layer Operating System . . . . . 16
4.2 Layer Software Dependencies . . . . . 16
4.3 Websockets Subsystem . . . . . 16
4.4 Stream Server Subsystem . . . . . 17
4.5 Web Data Handler Subsystem . . . . . 17
4.6 Database Subsystem . . . . . 18
4.7 Play Handler Subsystem . . . . . 19

**5 Frontend Layer Subsystems** **21**
5.1 Layer Hardware . . . . . 21
5.2 Layer Operating System . . . . . 21
5.3 Layer Software Dependencies . . . . . 21
5.4 Login Subsystem . . . . . 21
5.5 Dashboard Subsystem . . . . . 22
5.6 View Machines Subsystem . . . . . 23
5.7 In-claw State Subsystem . . . . . 24
5.8 Admin Subsystem . . . . . 24
5.9 Buy Tokens Subsystem . . . . . 25
5.10 Edit Profile Subsystem . . . . . 26

**6 Appendix A** **28**

## List of Figures


1 System architecture . . . . . 6
2 Power Supply subsystem diagram . . . . . 7
3 E-Stop subsystem diagram . . . . . 7
4 Camera subsystem diagram . . . . . 8
5 Computer subsystem diagram . . . . . 9
6 Light Controller Subsystem diagram . . . . . 11
7 Microcontroller Unit subsystem diagram . . . . . 12
8 Prize Sensor subsystem diagram . . . . . 13
9 Gantry Motherboard subsystem diagram . . . . . 14
10 Claw subsystem diagram . . . . . 15
11 Websockets subsystem diagram . . . . . 16
12 Stream Server subsystem diagram . . . . . 17
13 Web Data Handler subsystem diagram . . . . . 18
14 Database subsystem diagram . . . . . 19
15 Play Handler subsystem diagram . . . . . 20
16 Login subsystem description diagram . . . . . 21
17 Dashboard subsystem diagram . . . . . 22
18 View Machines subsystem diagram . . . . . 23
19 In Claw State subsystem diagram . . . . . 24
20 Admin subsystem diagram . . . . . 25
21 Buy Tokens subsystem diagram . . . . . 25
22 Edit Profile subsystem diagram . . . . . 26


## List of Tables

# 1 Introduction

This document presents the detailed design of the Clawser system, an IoT-enabled claw machine operatable remotely via a web-based application. The project integrates a camera-equipped claw cabinet with cloud-connected software to allow users to interact with a real claw machine via virtual tokens without being physically present. By enabling wireless control, live video streaming, and remote gameplay, the system expands the accessibility and potential audience of traditional claw machines while providing end users the convenience of remote play.

The design decisions described in this document are informed by the project's System Requirement Specification (SRS) and Architectural Design Specification (ADS) documents. The SRS defines the functional and non-functional goals of the system, including user interaction, remote control, and system constraints, while the ADS introduces the layered architecture and data flows that guide the system's structure. This document builds upon those foundations by elaborating on the internal implementation details of each layer and subsystem.

# 2 System Overview

The Clawser system follows a three-layer architecture—Cabinet, Cloud, and Frontend—that together enable remote, real-time interaction with an IoT-enabled claw machine. Data flows bidirectionally: control commands propagate downward from the user to the machine, while live video streams, cabinet status, and gameplay results propagate upward to keep users engaged and informed. Hardware control is encapsulated at the cabinet level, while coordination logic is abstracted into the cloud, enabling secure, scalable, and accessible remote gameplay without requiring users to be physically present at the machine.

At a high level, user actions originate in the Frontend Layer, are routed through the Cloud Layer for validation, coordination, and state management, and are ultimately executed in the Cabinet Layer as physical movements and sensory feedback. The Frontend Layer provides the user-facing web interface, handling authentication, machine selection, queue visibility, and live gameplay controls. All frontend interactions are mediated by the Cloud Layer, which acts as the system's central coordinator—managing play sessions, player queues, prize histories, and relaying validated control inputs to the machine. The Cabinet Layer forms the physical foundation of the system, consisting of embedded hardware and mechanical components responsible for real-time claw movement, sensor feedback, and video capture.

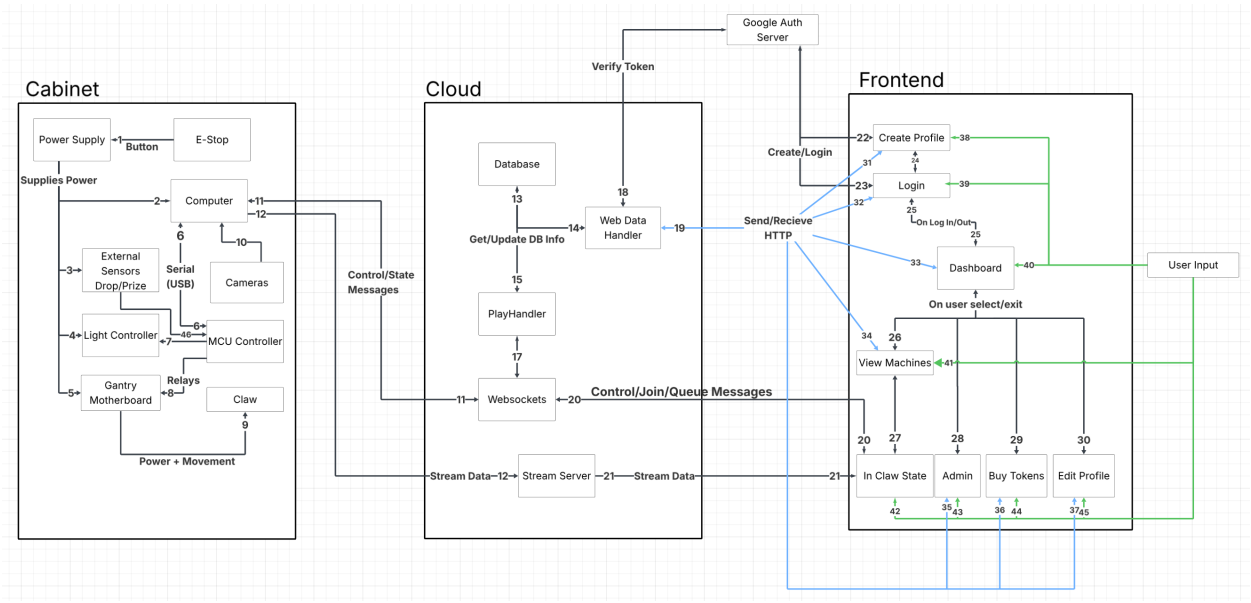



Figure 1: System architecture

# 3 Cabinet Layer Subsystems

The Cabinet Layer is responsible for all physical operation and real-time feedback of the claw machine, combining embedded hardware control with locally executed software running on an onboard computer. This layer interfaces directly with the Cloud Layer, receiving validated control commands and returning cabinet status updates and live video streams. Its design integrates commercial off-the-shelf hardware components with custom control software to ensure reliable and responsive machine operation.

## 3.1 Layer Hardware

The cabinet layer is a 81x84x142cm cabinet supported by 30x30mm aluminum extrustions with two acrylic panels for the front and side planes opposite two hardwood panels. The middle and bottom of the cabinet planes are constructed with hardwood panels for an enclosed supported play area and for housing cabinet hardware.

## 3.2 Layer Operating System

The Microsoft Windows 10 operating system is solely used for this layer.

## 3.3 Layer Software Dependencies

Layer software dependencies include LogiTune, OBS Studio... (aggregate the rest of the software + libraries listed in the subsystems of this layer)

## 3.4 Power Supply Subsystem

This subsystem is a piece of hardware that distributes power to all of the electronic components that require it.

### 3.4.1 Subsystem Hardware

The harware involved is a power strip that receives 120VAC from a receptacle that passes through the E-Stop and distributes it along a parallel line of power outlets.

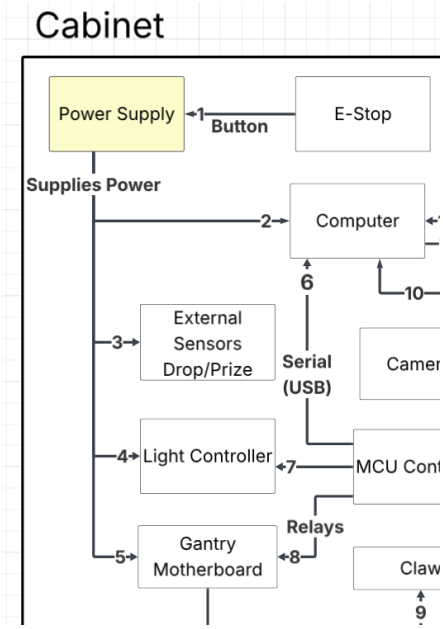



Figure 2: Power Supply subsystem diagram

### 3.4.2 Subsystem Operating System

The power supply subsystem does not have an operating system.

### 3.4.3 Subsystem Software Dependencies

As the power supply subsystem is a hardware component, software dependencies are not applicable.

### 3.4.4 Subsystem Programming Languages

The power supply subsystem does not use any programming languages to function.

### 3.4.5 Subsystem Data Structures

Data structures are inapplicable to the power supply subsystem.

### 3.4.6 Subsystem Data Processing

There is no data processing needed for this subsystem as it is hardware dependent.

## 3.5 E-Stop Subsystem

This is a physical fail-safe mechanism that immediately cuts power to the cabinet upon triggered to prevent possible injury and/or damage.

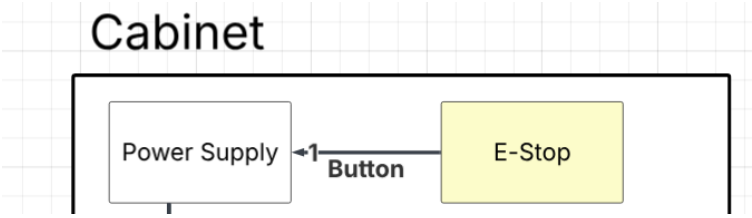



Figure 3: E-Stop subsystem diagram

### 3.5.1 Subsystem Hardware

The E-Stop is a pushable toggle that cuts the 120VAC feeding to the power supply. When triggered, the toggle must be reset in order to close the power supply circuit and restore power.

### 3.5.2 Subsystem Operating System

The emergency stop subsystem does not have an operating system it relies on.

### 3.5.3 Subsystem Software Dependencies

As it is hardware dependent, the emergency stop subsystem does not have software dependencies.

### 3.5.4 Subsystem Programming Languages

There are no programming languages used for this subsystem.

### 3.5.5 Subsystem Data Structures

No data structures are used on this subsystem.

### 3.5.6 Subsystem Data Processing

Although the emergency stop subsystem uses power to toggle supply, but otherwise does not have data processing.

## 3.6 Camera Subsystem

This subsystem is a set of video webcams mounted at different locations on the cabinet to provide users with video feeds of different views/angles of the physical game as they play.

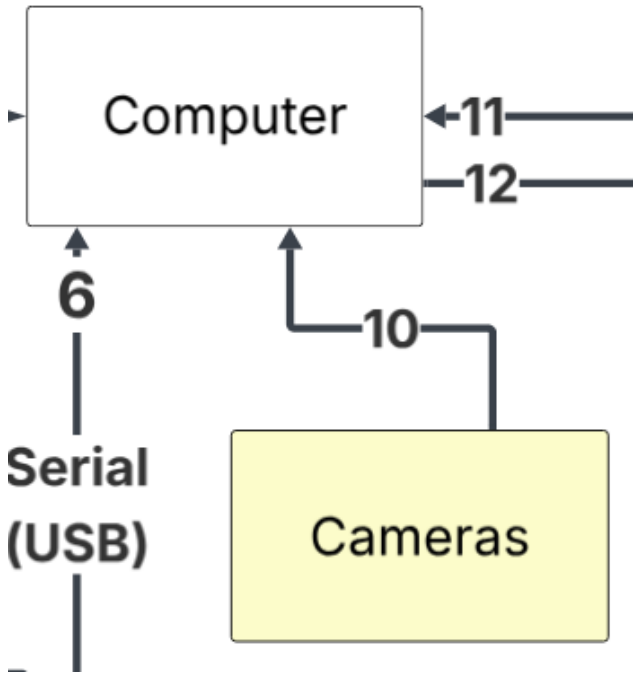



Figure 4: Camera subsystem diagram

### 3.6.1 Subsystem Hardware

Three Logitech Brio 101 webcams are placed and angled towards the front view of cabinet, the side view, and on the claw of the cabinet.

### 3.6.2 Subsystem Operating System

The Logitech Brio cameras are natively supported on the operating systems of the host computer, therefore utilizes Windows 10.

### 3.6.3 Subsystem Software Dependencies

LogiTune and Sync Portal are Logitechs proprietary device managers utilized for the Brio 101 webcams.

### 3.6.4 Subsystem Programming Languages

The camera subsystem for the cabinet is hardware focused, so it is not dependent on a programming language

### 3.6.5 Subsystem Data Structures

Each webcam in the subsystem provides its own video data over individual USB connection, managed by the Computer subsytem.

### 3.6.6 Subsystem Data Processing

A UVC stream from each camera sent to OBS to encode as a WebRTC WHIP stream.

## 3.7 Computer Subsystem

This subsystem is a small desktop computer that contains the software to control and read input from the Microcontroller Unit (MCU), manage webcam feeds, and create a single video stream to view. This subsystem connects to the Internet to receive and send websocket requests/commands for operation of the claw.

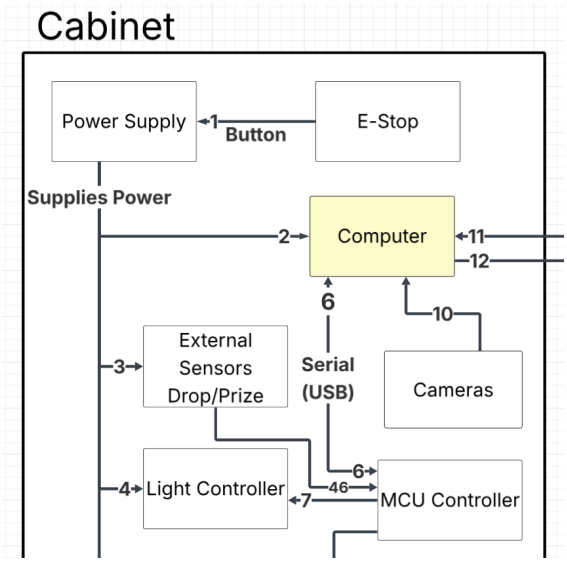



Figure 5: Computer subsystem diagram

### 3.7.1 Subsystem Hardware

A Dell Optiplex 7070 with an Intel i7-9700 and 16GB of RAM is mounted on the inside bottom of the cabinet.

### 3.7.2 Subsystem Operating System

The computer solely uses the Microsoft Windows 10 operating system.

### 3.7.3 Subsystem Software Dependencies

The Arduino framework is used to develop and use the MCU on the computer. The Arduino IDE is used to write C/C++ style code, which is then compiled to a binary to flash on the MCU. OBS Studio aggregates and prioritizes camera feeds according to websocket commands The pyserial library is used to communicate gantry motherboard data with the computer. The websockets library is used to initiate connections to the server and wait on action.

### 3.7.4 Subsystem Programming Languages

Python3 is used to listen and react to websocket events, along with communication with the MCU.

### 3.7.5 Subsystem Data Structures

A serial communication with the MCU is used, which uses integers to dictate claw movement. JSON data is transmitted between the server through the websocket to communicate user inputes and cabinet events. OBS Studio outputs a WebRTC-HTTP ingestion protocol (WHIP) stream to the server, encoded according to the H.264 codec.

### 3.7.6 Subsystem Data Processing

The computer system communicates with the cloud layer via websockets utilizing unsigned integer bits encoded according to UTF-8. Incoming input dictates user inputs and camera views, which are then communicated to the MCU and OBS. A status code will be returned to the cloud layer to indicate whether the cabinet is ready, in a game, or if the player won/loss.

## 3.8 Light Controller Subsystem

The Light Controller Subsystem provides internal illumination to improve visibility during gameplay. It supplies stable power to the cabinet lighting and, if supported by the selected controller, allows the lighting to be switched on/off or dimmed to match environmental conditions.

### 3.8.1 Subsystem Hardware

This subsystem consists of LED strips mounted along interior cabinet edges and an inline controller used for powering and optionally modulating the lights. The lighting system operates from mains power via the power supply and computer USB ports, and is rated for the cabinet environment at 5V 2A.

### 3.8.2 Subsystem Operating System

The light controller subsystem is not reliant on an operating system.

### 3.8.3 Subsystem Software Dependencies

The light controller is hardware focused rather than software focused, so there are no software dependencies.

### 3.8.4 Subsystem Programming Languages

As the light controller subsystem does not have software components, it does not have programming languages it uses.

### 3.8.5 Subsystem Data Structures

There are no data structures used in this subsystem.

### 3.8.6 Subsystem Data Processing

Without any data structures, the subsystem does have to do any data processing.

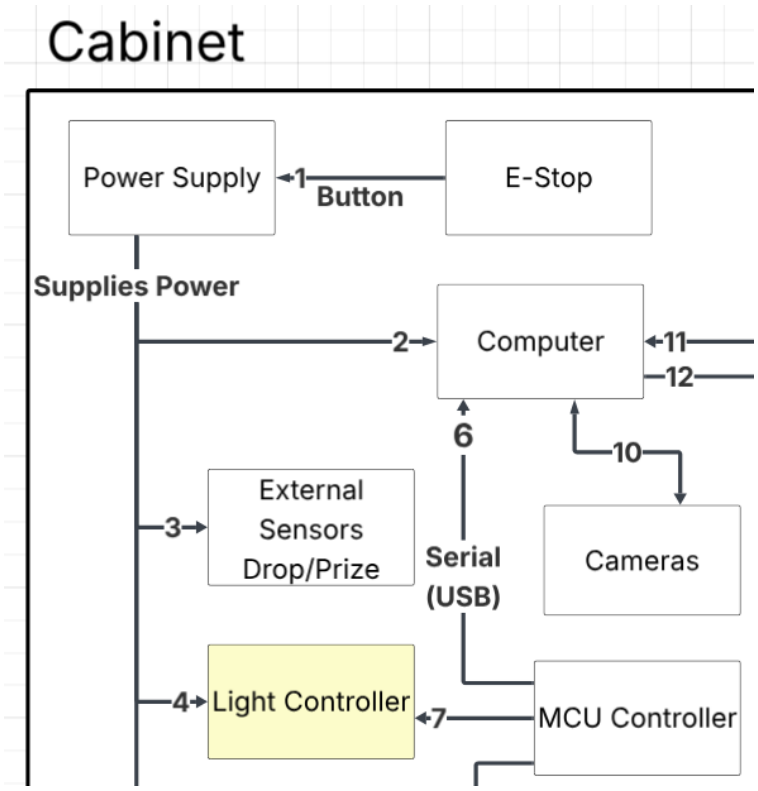



Figure 6: Light Controller Subsystem diagram

## 3.9 Microcontroller Unit Subsystem

The Microcontroller unit sends signals to the gantry motherboard's joystick, coin insert, and claw drop button pins for user movement and action feedback.

### 3.9.1 Subsystem Hardware

A RP2040-mini board is wired with the board pins. A USB connection is then used between the MCU and computer to transfer data.

### 3.9.2 Subsystem Operating System

N/A

### 3.9.3 Subsystem Software Dependencies

The program flashed to the MCU was made and compiled using the Arduino framework.

### 3.9.4 Subsystem Programming Languages

The programming language used for the MCU is the Arduino version of C/C++ v11.

### 3.9.5 Subsystem Data Structures

A serial connection is created through the USB line and data flows at a 115200 baud rate.

The MCU and the gantry motherboard are connected via a 115200 baud rate serial connection through the built in Serial-to-USB on the RP2040 chip.

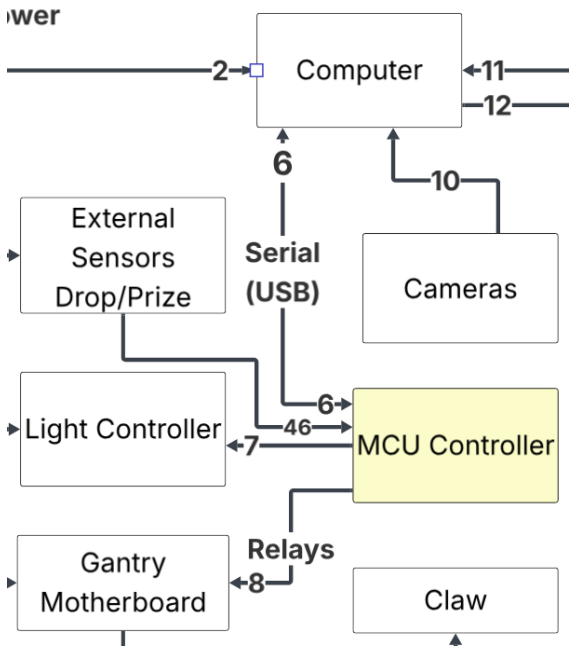



Figure 7: Microcontroller Unit subsystem diagram

#### 3.9.6 Subsystem Data Processing

The MCU listens for a new line sent through the serial port and executes a claw direction based on the integer received. It then simulates signals to the Gantry Motherboard of an up, down, left, or right movement was recieved along with button press for claw drop and coin insert for game start.

### 3.10 Prize Sensor Subsystem

This system uses computer vision based detection to determine if a prize has been won by the user. A camera is attached above the prize collection block and looks at markers placed in the prize zone to determine if a prize is in the collection block on top of the markers blocking camera detection.

#### 3.10.1 Subsystem Hardware

This system uses a camera that is mounted at the top of the cabinet, facing down into the prize collection block. It is the same Logitech Brio camera utilized in the camera subsystem.

#### 3.10.2 Subsystem Operating System

The operating system used in the subsystem is Windows 10.

#### 3.10.3 Subsystem Software Dependencies

The opencv-python and numpy libraries are used to run computer vision tasks that detect the markers.

#### 3.10.4 Subsystem Programming Languages

This system is written in the Python 3 programming language.

#### 3.10.5 Subsystem Data Structures

This system provides a boolean response to other systems requiring the status of the prize zone (empty or prize detected).

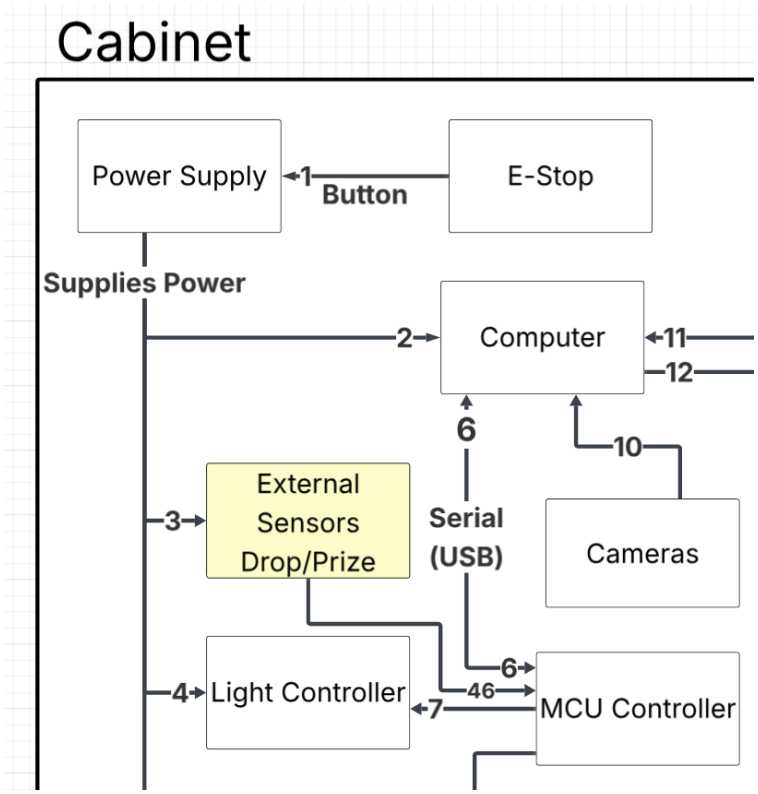



Figure 8: Prize Sensor subsystem diagram

#### 3.10.6 Subsystem Data Processing

To determine if a prize is won, the system takes an image after gameplay, when the claw is over the prize zone. That image is then used by OpenCV to see if it can find all 5 prize zone markers on the image. If it cannot, that indicates that the user has won and their prize is blocking the markers from being read. This result is output as a boolean.

### 3.11 Gantry Motherboard Subsystem

This is a manufacturer-produced claw game motherboard equipped with the logic to run a claw machine.

#### 3.11.1 Subsystem Hardware

The game motherboard provides physical dials and switches to control game logic and has pin inputs for various cabinet components (joysticks, buttons, coin sensors, displays, etc.). The motherboard also contains three potentiometers that can be adjusted to vary claw movement as well as grip strength. It also provides 24V power to the claw and controls its motors for movement.

#### 3.11.2 Subsystem Operating System

The gantry motherboard is a hardware component in the system, therefore the subsystem is not reliant on an operating system.

#### 3.11.3 Subsystem Software Dependencies

As the gantry motherboard does not have software components used in this subsystem, no software dependencies are needed.

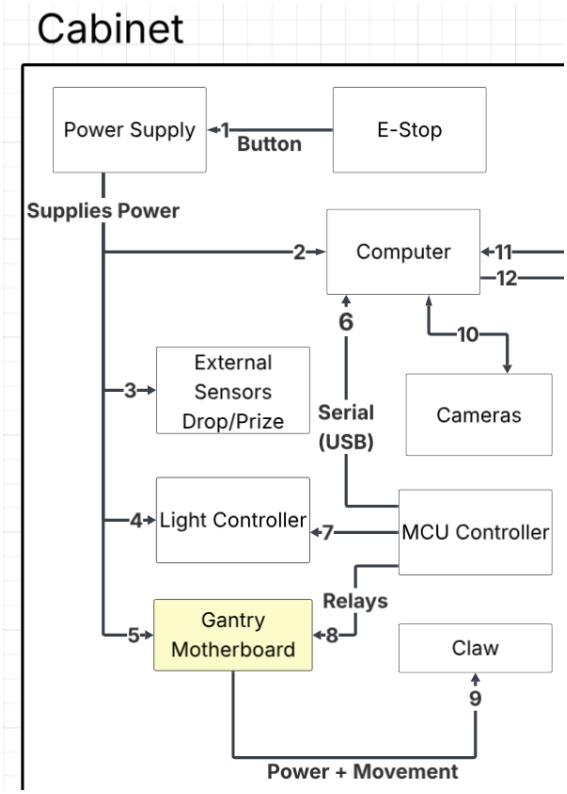



Figure 9: Gantry Motherboard subsystem diagram

### 3.11.4 Subsystem Programming Languages

Programming languages are not applicable to the subsystem.

### 3.11.5 Subsystem Data Structures

The motherboard has its own prefabricated internal logic. The MCU closes pins to the motherboard's GND to simulate movement of a joystick and a button press.

### 3.11.6 Subsystem Data Processing

N/A

## 3.12 Claw Subsystem

The claw is a physical component of the cabinet system. It is responsible for grabbing and dropping prizes inside the claw machine cabinet.

### 3.12.1 Subsystem Hardware

The claw subsystem is a physical grapple to move within the confines of the cabinet space. It is responsible for grasping and releasing prizes inside the machine by receiving input from the gantry motherboard.

### 3.12.2 Subsystem Operating System

The claw subsystem by itself is a hardware subsystem, therefore it does not rely on an operating system.

### 3.12.3 Subsystem Software Dependencies

The subsystem does not use any software on its own, therefore it does not have any software dependencies.

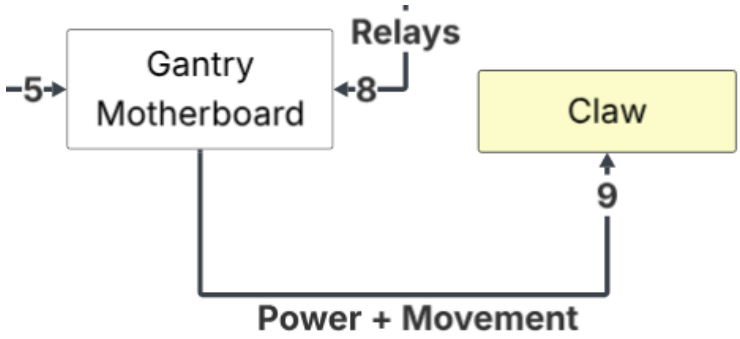



Figure 10: Claw subsystem diagram

#### 3.12.4 Subsystem Programming Languages

There are no programming languages used for this subsystem.

#### 3.12.5 Subsystem Data Structures

As the claw does not use software, it is not using any data structures.

#### 3.12.6 Subsystem Data Processing

The claw receives input from MCU via the gantry motherboard, dropping, grabbing, and releasing prizes on command.

# 4 Cloud Layer Subsystems

The Cloud Layer facilitates communication between the Hardware Layer and the Frontend Layer. This Layer is responsible for managing the player database, connecting the video stream, validating sign-on statuses, and ensuring that user inputs from the Frontend Layer are relayed to the Cabinet Layer so that the claw can be moved without having physical access to the controls of the claw machine. Token creation and validation also allows to ensure we can identify users securely and not waste resources.

## 4.1 Layer Operating System

Debian 12 Bookworm is used for all three Google Cloud virtual machines.

## 4.2 Layer Software Dependencies

MediaMTX is used for the stream server. The database server runs MariaDB, an open-source fork of MySQL. The API server is written in Go and utilizes several libraries: Gorilla Websockets for websocket management, Gorilla Mux for HTTP server functionality, Go-Sql-Driver/mysql for providing a MySQL API, Golang-JWT for creating and validating JSON Web Tokens (JWTs), Google-UUID for unique ID creation, and the Google Golang API for the validation and retrieval of Google JWTs.

## 4.3 Websockets Subsystem

The Websocket subsystem provides a method of communication between user clients and game machines. Game machines can create websocket rooms, which host games to host games for clients can connect to. Client can connect to the websocket rooms in order to communicate with the machine. This includes sending messages for moving a claw, or receiving a message to tell you your place in the queue on the server. Ultimately, all gameplay-related communication (aside from video streams) is facilitated via websockets.

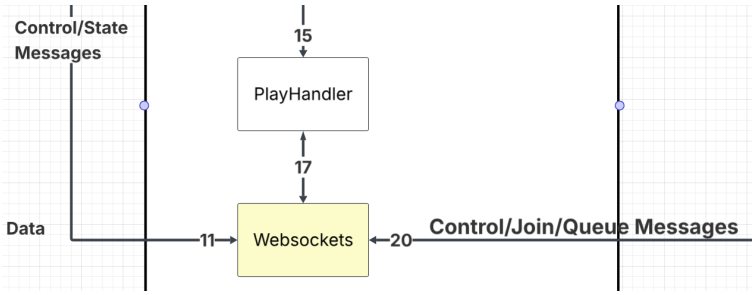



Figure 11: Websockets subsystem diagram

### 4.3.1 Subsystem Hardware

This subsystem runs on an E2-Small instance in Google Cloud.

### 4.3.2 Subsystem Operating System

The E2-Small instance runs on Debian 12 Bookworm.

### 4.3.3 Subsystem Software Dependencies

This subsystem's dependencies include Gorilla Websocket, Gorilla Mux, and Golang-JWT.

### 4.3.4 Subsystem Programming Languages

This subsystem is written in Golang (or Go for short).

#### 4.3.5 Subsystem Data Structures

As the Websocket protocol allows for transfer of data through strings or bytes, messages are sent through the JSON format as a string. Messages may also be sent in byte form. However, there is no defined standard schema for the JSON format in order to maintain support for a variety of games. As such, it is up to the machine hosting the game to interpret and handle these messages.

#### 4.3.6 Subsystem Data Processing

Nothing to note worth here, other than using JSON and bytes for messages.

### 4.4 Stream Server Subsystem

The Stream Server subsystem handles the transmission and conversion of video formats so that the clients can recieve low-latency video stream. Because streaming video is network- and hardware-intensive, the routes are either password-protected or authorized via client-provided JWTs.

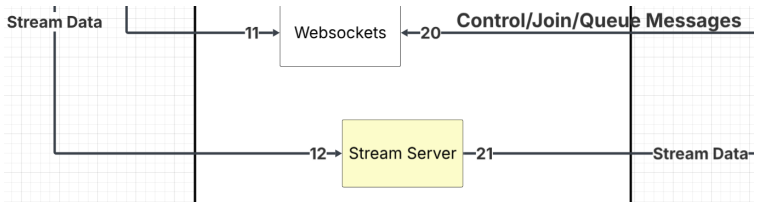



Figure 12: Stream Server subsystem diagram

#### 4.4.1 Subsystem Hardware

This subsystem runs on an E2-Medium instance on Google Cloud.

#### 4.4.2 Subsystem Operating System

The E2-Small instance runs on Debian 12 Bookworm.

#### 4.4.3 Subsystem Software Dependencies

The websocket server runs MediaMTX.

#### 4.4.4 Subsystem Programming Languages

MediaMTX is written in Golang, while the server configuration is specified in the YAML file format.

#### 4.4.5 Subsystem Data Structures

WHIP, WHEP and WebRTC protocols are used to deliver video streams to clients with low latency.

#### 4.4.6 Subsystem Data Processing

The stream server is optimized for converting WHIP video stream input from a claw machine into WHEP/WebRTC output. This output is then served to viewers.

### 4.5 Web Data Handler Subsystem

The Web Data Handler Subsystem is a webserver that handles backend information processing, and features. Tasks such as logging in, getting a list of connected machines, purchasing tokens, editing and viewing address information are handled here.

#### 4.5.1 Subsystem Hardware

This subsystem runs on an E2-Small instance on Google Cloud.

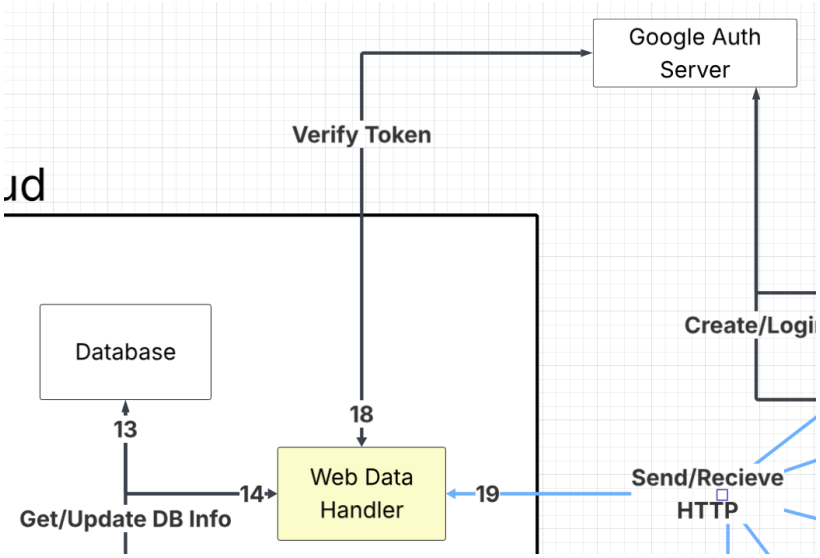



Figure 13: Web Data Handler subsystem diagram

### 4.5.2 Subsystem Operating System

The E2-Small instance runs on Debian 12 Bookworm.

### 4.5.3 Subsystem Software Dependencies

Web Data Handler software dependencies include Gorilla Mux for HTTP server functionality, Go-Sql-Driver/mysql for providing a MySQL API, Golang-JWT for creating and validating JWTs, Google-UUID for unique ID creation, and the Google Golang API for the validation and retrieval of Google JWTs.

### 4.5.4 Subsystem Programming Languages

The web data handler is written in Go. Posts and responses are all in the JSON file format.

### 4.5.5 Subsystem Data Structures

All actions in this subsystem require the creation or validation of a JWT. Authenticating a login with a Google account requires validating a JWT authored by Google and returning a JWT in response.

### 4.5.6 Subsystem Data Processing

The web data hander subsystem ensures the incoming and outgoing of all data sent over HTTP. Client requests receive JWTs over HTTP for verification and indentification. Google authentication requests the subsystem validates using the Google Cloud API.

## 4.6 Database Subsystem

The Database Subsystem Layer is responsible for the storage and retrival of users, machines, token shop products, token counts, addresses, profile information, prize history, and error data. Data is stored in a relational database; as such, most schemas have a unique identifier for ease of joining tables.

### 4.6.1 Subsystem Hardware

This subsystem runs on a E2-Small instance on Google Cloud.

### 4.6.2 Subsystem Operating System

This subsystem runs Debian 12 Bookworm.

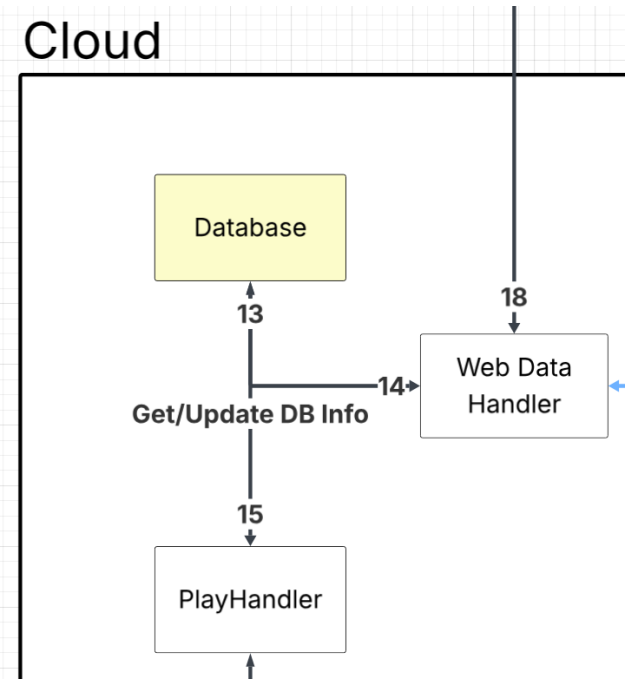



Figure 14: Database subsystem diagram

#### 4.6.3 Subsystem Software Dependencies

MariaDB is used as the relational database for this subsystem. It is open-source, but functionally equivalent to MySQL.

#### 4.6.4 Subsystem Programming Languages

SQL is the language used for formatting datbase queries.

#### 4.6.5 Subsystem Data Structures

The database contains multiple tables tracking information related to players, wins, prizes, and machines. The "prize" table is used for storing and managing prizes, and associates each prize type with a unique ID, a quantity, and a description. The "player" table includes the names, username, unique ID, and tokens of each player. The "machines" table stores the name of the claw machine, token cost per play, and status. The schema is set up for efficient update and creation of tables.

#### 4.6.6 Subsystem Data Processing

This subsystem is mostly used for data retrievals and storage, rather than data transformation.

### 4.7 Play Handler Subsystem

This subsystem manages in-game specifics and act more as a post man. It mostly operates by managing a queue and forwarding packets of information between clients. For instance, for the claw machine, it is in charge of managing whose turn it is, forwarding messages to and from the right person, using the Websocket subsystems and sending messages indicating order in the queue along with other system messages. For instance, before allowing the player to play the machine, it is in charge of deducting the token to play.

#### 4.7.1 Subsystem Hardware

This subsystem runs on a E2-Small instance on Google Cloud.

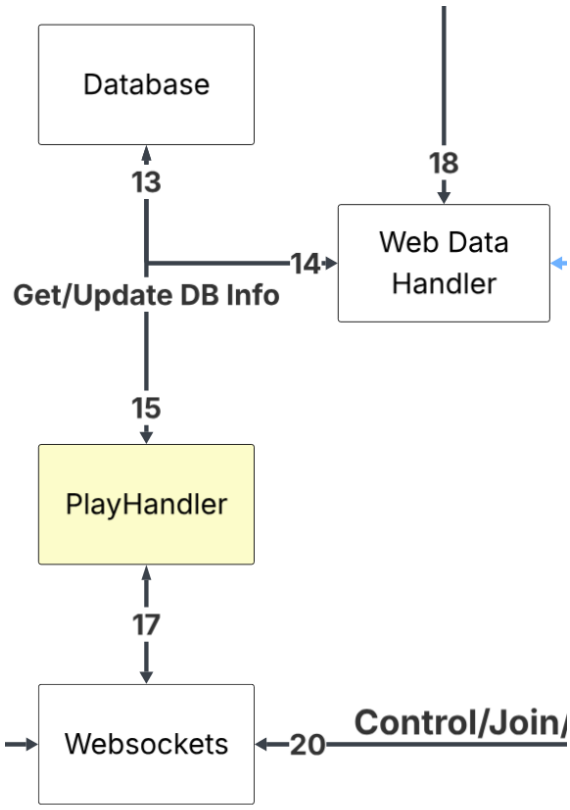



Figure 15: Play Handler subsystem diagram

#### 4.7.2 Subsystem Operating System

The operating system used by this system is Debian 12 Bookworm.

#### 4.7.3 Subsystem Software Dependencies

The Play Handler subsystem relies on a Gorilla WebSocket library in order to establish WebSocket connections. HTTP client libraries from Go are also used to communicate with the Web Data Handler.

#### 4.7.4 Subsystem Programming Languages

The Play Handler uses Golang.

#### 4.7.5 Subsystem Data Structures

Due to the lack of a native queue data structure in Golang, we created our own thread safe queue with index based removal by making a object using Go's slice data structure and a read write mutex. This was done by simply making a struct with the mutex and the slice, and making functions that do the intended queue functionality.

#### 4.7.6 Subsystem Data Processing

This subsystem should process messages intended for the server and not the machine hosting the game. For instance, messages indicating chat between clients should be sent to connected clients. Messages indicating control of the machine should be sent to the connected machine. Messages indicating joining, removing the queue should not be forwarded to any machine and instead processed by the server itself and handled accordingly.

# 5 Frontend Layer Subsystems

This section describes the Frontend Layer software design and runtime environment. The Clawser client is implemented as an Expo-managed React Native application using TypeScript, delivering a consistent experience on iOS and Android [and optionally via Expo web]. User actions are sent to the Cloud Layer over HTTP(S) and WebSocket as implemented in the deployment; [document whether production uses TLS end-to-end or current cleartext configuration]. Live video during play is shown using a WebView loading the stream server URL for the selected cabinet. Global state, authentication persistence, and navigation are handled with [Zustand, Expo Secure Store, and React Navigation] as described in the subsystems below.

## 5.1 Layer Hardware

No dedicated frontend hardware is required. The application runs on standard smartphones and tablets [and supported simulators/emulators for development]. All physical claw hardware is accessed only through the Cloud and Cabinet layers.

## 5.2 Layer Operating System

The client targets iOS and Android via React Native. Minimum OS versions: e.g. iOS, Android API fill from app.json / store requirements. Development builds may use Expo dev client and EAS Build [if applicable].

## 5.3 Layer Software Dependencies

Native modules and Expo config plugins (e.g. Google Sign-In, Secure Store, build properties) are declared in app.json / app.config. Additional frontend frameworks or libraries may be used to manage user interface components, application state, and asynchronous communication, but all machine control logic and data persistence are handled externally by cloud services.

## 5.4 Login Subsystem

The Login Subsystem enables users to authenticate into the Clawser application using Google single sign-on. It presents the login interface, initiates authentication with Google services, and forwards the received authentication token to the Cloud Layer for validation. Based on the verification response, the subsystem either transitions the user to the Dashboard or displays an error state.

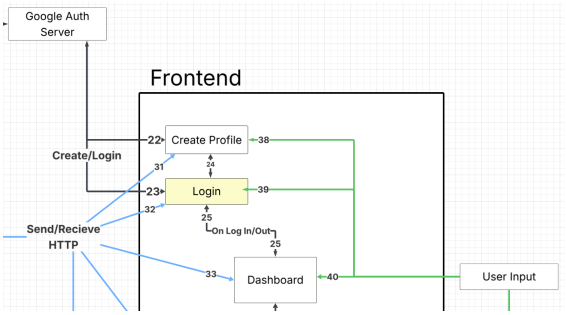



Figure 16: Login subsystem description diagram

### 5.4.1 Subsystem Hardware

No hardware components are associated with this subsystem.

### 5.4.2 Subsystem Operating System

The login system runs on operating systems supported by mobile and desktop devices via the React Native runtime.

### 5.4.3 Subsystem Software Dependencies

Google Sign-In, Expo Secure Store, Expo Web Browser, JWT decode, REST client.

### 5.4.4 Subsystem Programming Languages

The programming language is in TypeScript as this is supported by the projects UI software framework, React Native.

### 5.4.5 Subsystem Data Structures

The data structures for the login page include authentication tokens (JWT), user session objects, and login state flags.

### 5.4.6 Subsystem Data Processing

The login system processes authentication responses, validates login state, and manages transitions to authenticated views.

## 5.5 Dashboard Subsystem

The Home screen is the default destination after login. It presents the primary Play entry (navigate to cabinet selection) [and any marketing/branding content]. Bottom navigation switches among Home, Prize, and Profile. Token balance shown in shop/cabinet flows is backed by UserDataStore and refreshed via `GET [BASE_URL]/api/tokens`[where used]. The stack navigator manages modal-style transitions to Shop, Play, Edit Profile, Edit Address, Report Error, Prize Tracking, and Cabinet Selection.

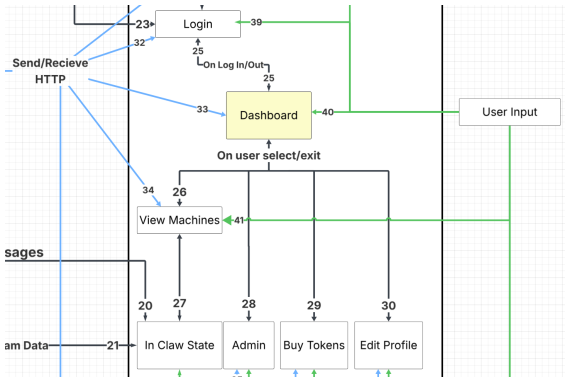



Figure 17: Dashboard subsystem diagram

### 5.5.1 Subsystem Hardware

No hardware components are associated with this subsystem.

### 5.5.2 Subsystem Operating System

The subsystem supports cross-platform execution through React Native.

### 5.5.3 Subsystem Software Dependencies

The dashboard system uses cloud API services, state management libraries, and UI navigation frameworks.

### 5.5.4 Subsystem Programming Languages

The subsystem uses React Native as it's UI framework which supports TypeScript as the main programming language.

### 5.5.5 Subsystem Data Structures

Data structures in the dashboard system include user account objects, navigation state, dashboard view models.

### 5.5.6 Subsystem Data Processing

Fetches user data from the Cloud Layer and updates the interface in response to real-time state changes.

## 5.6 View Machines Subsystem

The View Machines Subsystem allows users to browse available claw machines and observe their status, including queue length and availability. When a machine is selected, the subsystem sends a join request to the Cloud Layer and waits for authorization before transitioning to the gameplay state.

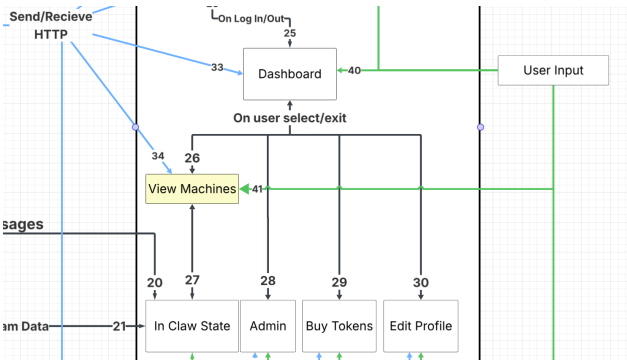



Figure 18: View Machines subsystem diagram

### 5.6.1 Subsystem Hardware

No hardware components are associated with this subsystem.

### 5.6.2 Subsystem Operating System

Runs on supported React Native platforms.

### 5.6.3 Subsystem Software Dependencies

The View Machines subsystem has software dependencies for cloud data APIs, WebSocket messaging services, UI rendering libraries.

### 5.6.4 Subsystem Programming Languages

The programming language for the system uses TypeScript as supported by React Native.

### 5.6.5 Subsystem Data Structures

Data structures for this subsystem include machine metadata objects, queue state records, user selection states.

### 5.6.6 Subsystem Data Processing

Retrieves machine lists, updates queue information, and manages state transitions based on server responses.

## 5.7 In-claw State Subsystem

The In-Claw State Subsystem manages live gameplay interactions during an active session. It displays the live video stream provided by Google Cloud streaming services, transmits user control commands via WebSockets, enforces session timing, and reflects real-time machine feedback. This subsystem ensures synchronized interaction between the user interface and the physical claw machine.

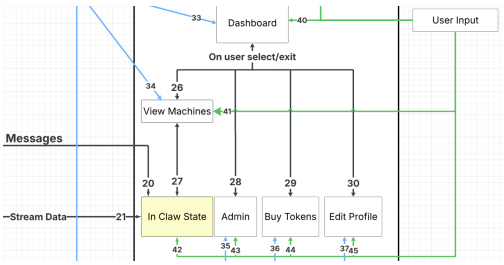



Figure 19: In Claw State subsystem diagram

### 5.7.1 Subsystem Hardware

No hardware components are associated with this subsystem.

### 5.7.2 Subsystem Operating System

The subsystem operating system is a cross-platform React Native execution environment.

### 5.7.3 Subsystem Software Dependencies

The In Claw State subsystem has WebSocket libraries, Google Cloud streaming services, real-time UI update frameworks dependencies.

### 5.7.4 Subsystem Programming Languages

The programming language used is TypeScript, supported by React Native.

### 5.7.5 Subsystem Data Structures

The In Claw State subsystem uses data structures such as control command messages, stream state objects,and session timers.

### 5.7.6 Subsystem Data Processing

The subsystem encodes user inputs into network messages, processes machine acknowledgments, and synchronizes video streams with gameplay controls.

## 5.8 Admin Subsystem

The Admin Subsystem provides authorized users with elevated access to monitor, manage, and configure claw machines within the Clawser platform. This subsystem functions as an operational control interface, allowing administrators to view machine status, perform maintenance actions, and manage system-level settings. Access to this subsystem is restricted through role-based authentication enforced by the Cloud Layer.

### 5.8.1 Subsystem Hardware

No hardware components are associated with this subsystem.

### 5.8.2 Subsystem Operating System

The subsystem operating system uses a cross-platform React Native execution environment.

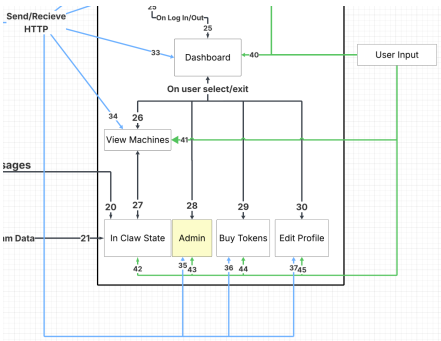



Figure 20: Admin subsystem diagram

### 5.8.3 Subsystem Software Dependencies

The admin subsystem relies on React Native UI components, HTTPS-based Cloud APIs, authentication and authorization services.

### 5.8.4 Subsystem Programming Languages

TypeScript is the programming language used in this subsystem as supported by React Native.

### 5.8.5 Subsystem Data Structures

The admin subsystem uses data structures for administrator role identifiers, machine status objects, configuration parameter records.

### 5.8.6 Subsystem Data Processing

Retrieves real-time machine data from the Cloud Layer, validates administrative actions, and transmits control or configuration updates securely to backend services.

## 5.9 Buy Tokens Subsystem

The Buy Tokens Subsystem enables authenticated users to purchase digital tokens used to access gameplay sessions. It presents available token packages, initiates purchase requests, and communicates with the Cloud Layer to update the user's token balance upon successful transaction completion. This subsystem ensures secure handling of purchase requests and immediate reflection of updated balances in the user interface.

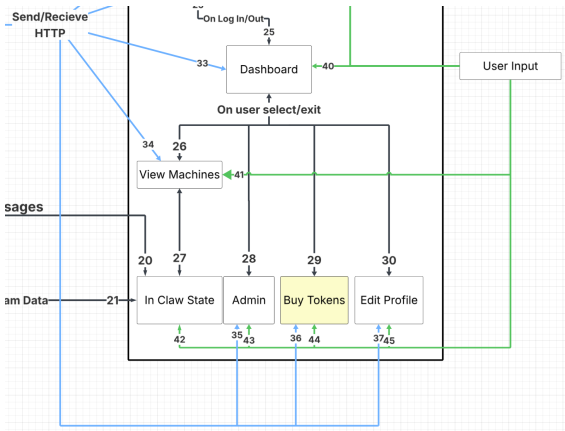



Figure 21: Buy Tokens subsystem diagram

### 5.9.1 Subsystem Hardware

No hardware components are associated with this subsystem.

### 5.9.2 Subsystem Operating System

The Buy Tokens subsystem operating system has a cross-platform React Native execution environment.

### 5.9.3 Subsystem Software Dependencies

Buy Tokens subsystem is dependent on React Native UI components, HTTPS-based Cloud APIs, authentication and authorization services.

### 5.9.4 Subsystem Programming Languages

The subsystem programming language used is TypeScript as supported by React Native.

### 5.9.5 Subsystem Data Structures

The Buy Tokens subsystem uses data structures such as token package objects, transaction request records, and user balance data.

### 5.9.6 Subsystem Data Processing

Validates purchase requests, sends transaction data to the Cloud Layer, and updates the frontend state based on backend confirmation responses.

## 5.10 Edit Profile Subsystem

The Edit Profile Subsystem allows users to view and modify personal account information such as display name, profile details, and account preferences. In addition, the subsystem provides navigation to other pages with related functions such as prize tracking, and interfaces with the Login Subsystem to support user logout. It provides a controlled interface for submitting profile updates, which are validated and securely transmitted to the Cloud Layer for persistence. The subsystem ensures that changes are reflected consistently across the application once confirmed.

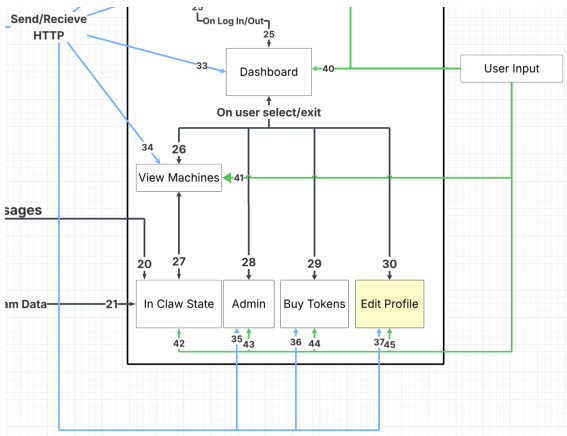



Figure 22: Edit Profile subsystem diagram

### 5.10.1 Subsystem Hardware

No hardware components are associated with this subsystem.

### 5.10.2 Subsystem Operating System

Runs on all supported operating systems through the React Native environment.

### 5.10.3 Subsystem Software Dependencies

The subsystem has the React Native form components, Cloud authentication services, HTTPS-based data APIs software dependencies.

### 5.10.4 Subsystem Programming Languages

The subsystem programming language is TypeScript which is supported by the project UI, React Native.

### 5.10.5 Subsystem Data Structures

Edit Profile uses data structures for user profile objects, editable field models, validation state flags.

### 5.10.6 Subsystem Data Processing

Validates user input, packages profile updates into secure requests, submits changes to the Cloud Layer, and refreshes the local application state upon confirmation.

# 6 Appendix A

# References